

Cours : Traitement d'images

Chapitre 3: Filtrage d'images

Objectifs

Ce cours s'intéresse aux techniques d'*amélioration des images numériques*, pour *augmenter la qualité* de leur rendu visuel, ou pour faciliter leur analyse. On cherche donc à atténuer, sinon supprimer une certaine *dégradation*. Celle-ci n'est pas forcément connue *a priori*, mais elle peut parfois être estimée *a posteriori*. On distinguera ici :

- les dégradations liées au *bruit* : $g(x) = f(x) + b(x)$ ou $g(x) = f(x)b(x)$ liées au capteur, à la quantification, à la transmission... On les traite en tirant parti des informations locales par le *filtrage*. Par différenciation, les techniques de filtrage permettent en outre de calculer ou amplifier les contrastes locaux.
- les dégradations *convolutives* : $g(x) = f(x).b(x)$ liées à un mouvement du capteur ou un défaut de mise au point. On les traite en inversant un opérateur linéaire, donc supposé connu : ce sont les techniques dites de *restauration* (voir cours *Restauration*).



bruit additif



bruit multiplicatif



flou de mise au point



flou de bougé

Plan du chapitre

1. Convolution Numérique
2. Filtrage par convolution
3. Filtres non linéaires (autre que convolution)

1. Convolution : définition

Convolution numérique

La convolution discrète est un outil permettant l'utilisation de **filtres linéaires** ou de **filtres de déplacements invariants**

L'équation générale de la convolution, notée $g(x)$, de la fonction d'origine $f(x)$ avec une fonction $h(x)$ est :

$$g(x) = f(x) * h(x) = \sum_{\forall k} h(x-k) f(k)$$

$f(x)$ est la fonction d'origine et $g(x)$ la fonction convoluée (résultat de la convolution)

- Dans notre cas, une image est vue comme une fonction mathématique

$h(x)$ est appelé *masque de convolution*, *noyau de convolution*, *filtre*, *fenêtre*, *kernel*, ...

Convolution

Soit I une image numérique.

Soit h une fonction de $[x_1, x_2] \times [y_1, y_2]$ à valeurs réelles.

La *convolution* de I par h est définie par :

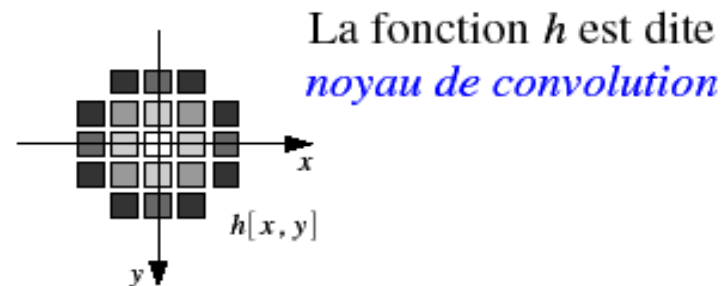
$$(I * h)[x, y] = \sum_{i=x_1}^{x_2} \sum_{j=y_1}^{y_2} h[i, j] \cdot I[x-i, y-j]$$

Propriétés de la convolution :

COMMUTATIVITÉ $h * g = g * h$

ASSOCIATIVITÉ $(h * g) * k = h * (g * k) = h * g * k$

DISTRIBUTIVITÉ / + $h * (g + k) = (h * g) + (h * k)$



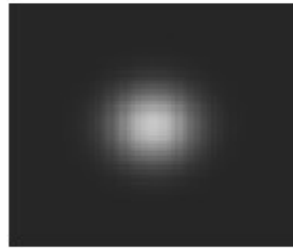
Les nouvelles valeurs du pixel sont calculées par *produit scalaire* entre le noyau de convolution et le *voisinage* correspondant du pixel.

Exemple de convolution 2D



Image d'origine

*



Filtre de convolution
(masque)

=

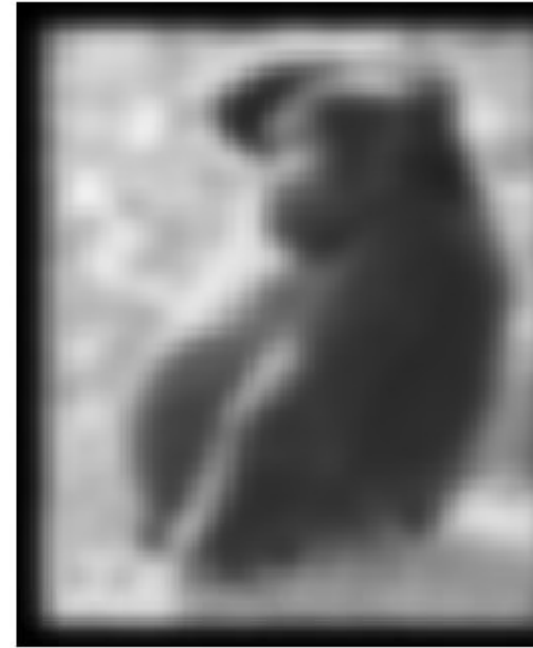


Image convoluée
(résultat)

Note : par convention pratique, la taille de l'image résultat est la même que celle de l'image d'origine

Convolution numérique discrète

- En pratique, la convolution numérique d'une image se fera par une **sommation de multiplications**
- Un filtre de convolution est une matrice (image) généralement (*mais pas toujours*) de **taille impaire et symétrique**
 - 3x3, 5x5, 7x7, ...

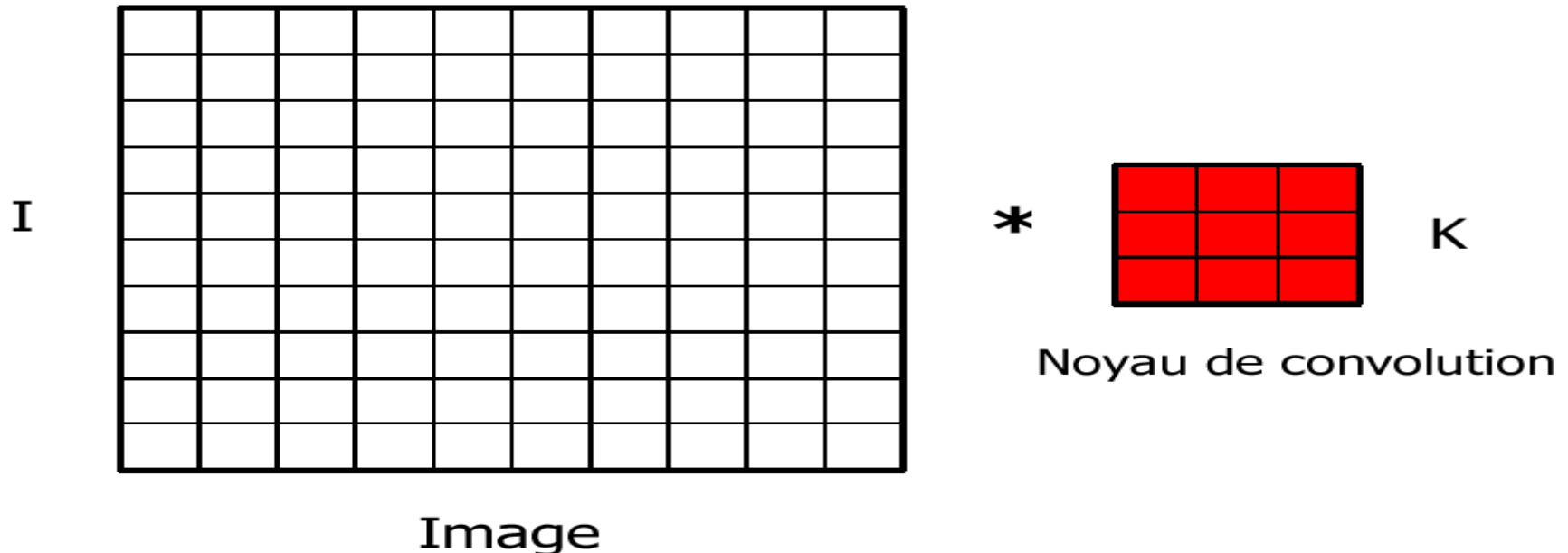
Convolution d'une image par un filtre 2D :

$$I'(i, j) = I(i, j) * \text{filtre}(i, j)$$

$$I'(i, j) = \sum_u \sum_v I(i-u, j-v) \cdot \text{filtre}(u, v)$$

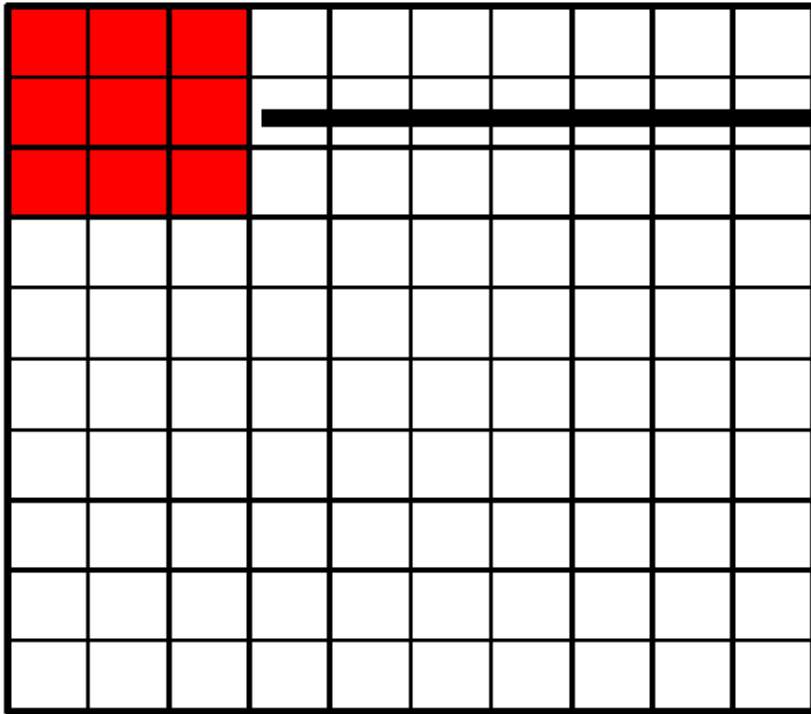
Convolution numérique

L'opération de convolution de l'image g par la fonction h , dite *noyau de convolution* consiste donc à remplacer la valeur de chaque pixel $g[x, y]$ par une combinaison linéaire de ses voisins, où le coefficient attaché à chaque voisin $(x - i, y - j)$ est déterminé par la valeur $h[i, j]$ du noyau de convolution.

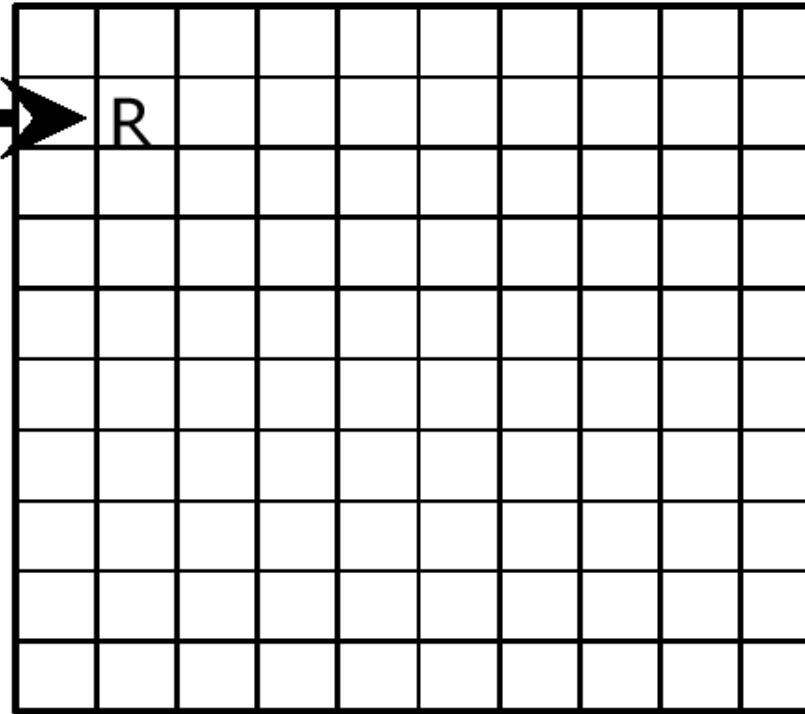


Convolution numérique $R=I*K$

K



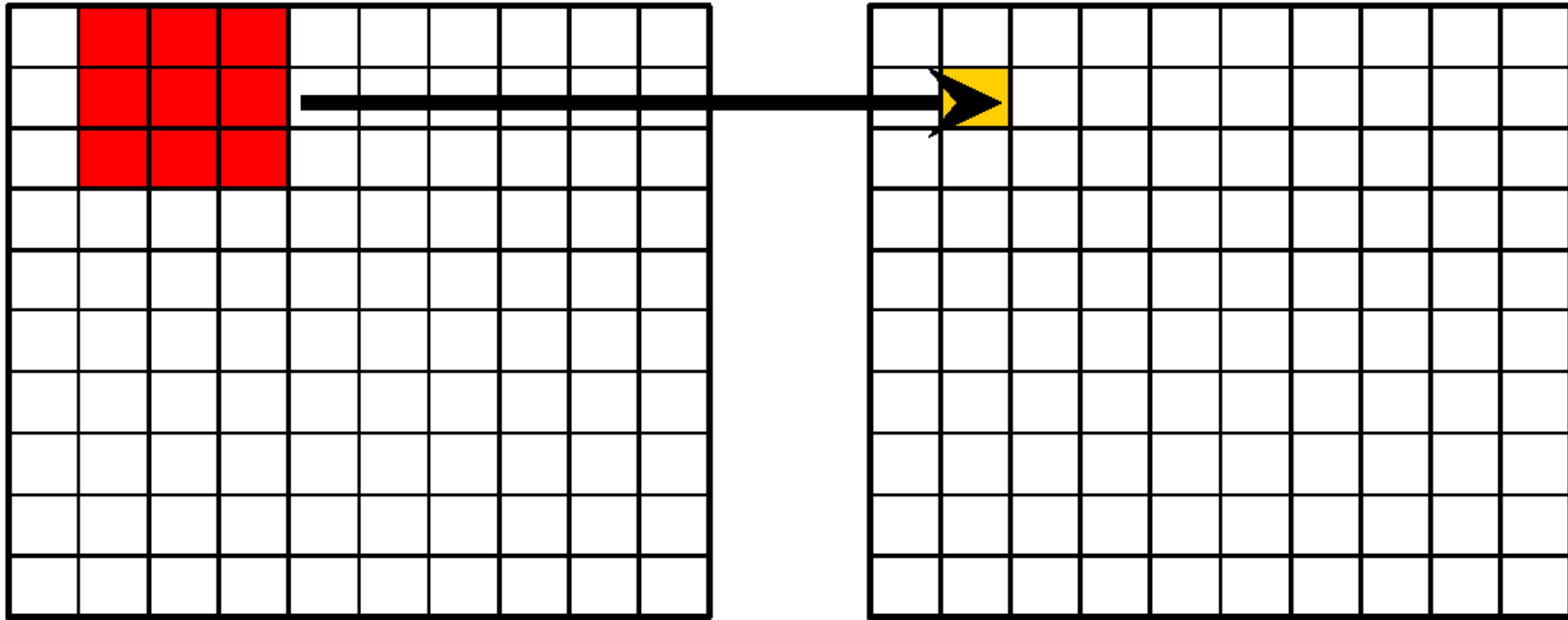
R



$$\begin{aligned} R(1,1) = & I(0,0) K(0,0) + I(1,0) K(1,0) + I(2,0) K(2,0) \\ & + I(0,1) K(0,1) + I(1,1) K(1,1) + I(2,1) K(2,1) \\ & + I(0,2) K(0,2) + I(1,2) K(1,2) + I(2,2) K(2,2) \end{aligned}$$

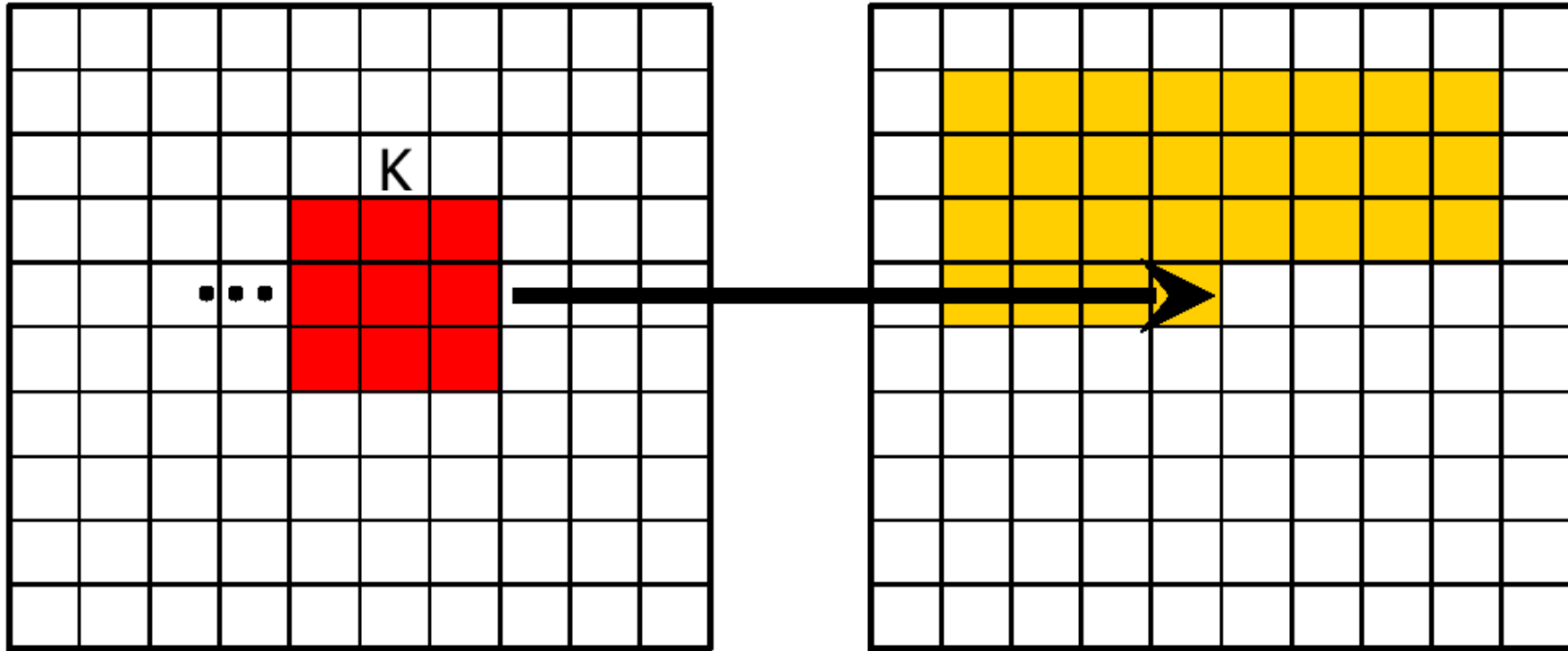
Convolution numérique $R=I*K$

K



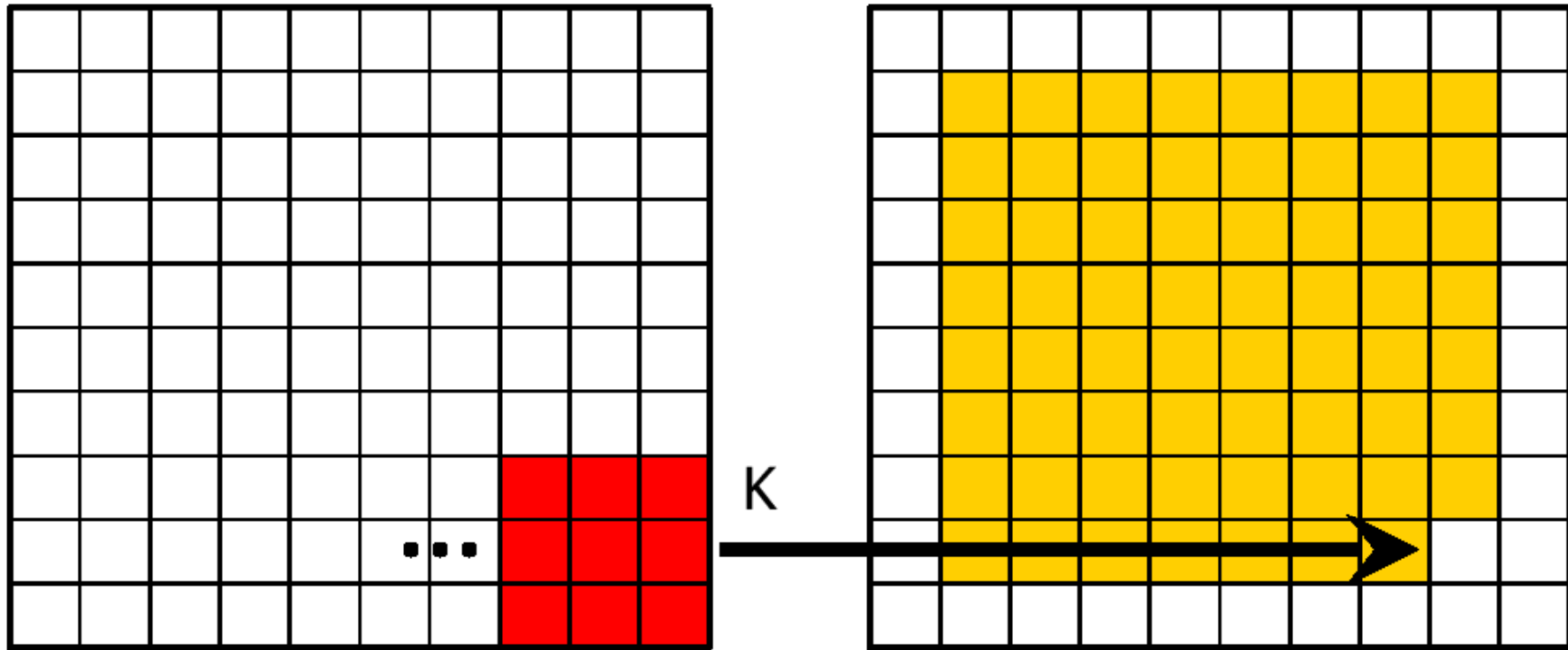
$$\begin{aligned} R(2,1) = & I(1,0) K(0,0) + I(2,0) K(1,0) + I(3,0) K(2,0) \\ & + I(1,1) K(0,1) + I(2,1) K(1,1) + I(3,1) K(2,1) \\ & + I(1,2) K(0,2) + I(2,2) K(1,2) + I(3,2) K(2,2) \end{aligned}$$

Convolution numérique $R=I*K$



$$\begin{aligned} R(x,y) = & I(x-1,y-1) K(0,0) + I(x, y-1) K(1,0) + I(x+1, y-1) K(2,0) \\ & + I(x-1,y) K(0,1) + I(x,y) K(1,1) + I(x+1,y) K(2,1) \\ & + I(x-1,y+1) K(0,2) + I(x,y+1) K(1,2) + I(x+1,y+1) K(2,2) \end{aligned}$$

Convolution numérique $R=I*K$



$$\begin{aligned}
 R(N-2, M-2) = & I(N-3, M-3) K(0,0) + I(N-2, M-3) K(0,1) + I(N-1, M-3) K(0,2) \\
 & + I(N-3, M-2) K(1,0) + I(N-2, M-2) K(1,1) + I(N-1, M-2) K(1,2) \\
 & + I(N-3, M-1) K(2,0) + I(N-2, M-1) K(2,1) + I(N-1, M-1) K(2,2)
 \end{aligned}$$

Convolution numérique

problème : Que faire avec les bords de l'image ?

- Mettre à zéro (0)
- Convolution partielle
 - Sur une portion du noyau
- Miroir de l'image
 - $f(-x,y) = f(x,y)$
- ... (pas de solution miracle)

?	?	?	?	?	?	?	?	?	?
?									?
?									?
?									?
?									?
?									?
?									?
?									?
?									?
?	?	?	?	?	?	?	?	?	?

Masque de convolution

Le masque de convolution représente un filtre linéaire permettant de modifier l'image

On divisera le résultat de la convolution par la somme des coefficients du masque

- Pour éviter de modifier la luminance globale de l'image, la somme des coefficients doit être égale à 1

2. Filtrage par convolution

deux types de filtrage spatial

- **Filtres passe-bas**

- Atténue le bruit et les détails (basses fréquences)
→ lissage

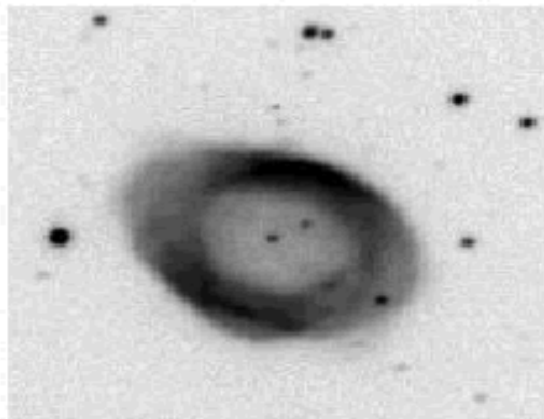


- **Filtres passe-haut**

- Accentue les détails et les contours (hautes fréquences)
→ accentuation

Filtre passe haut

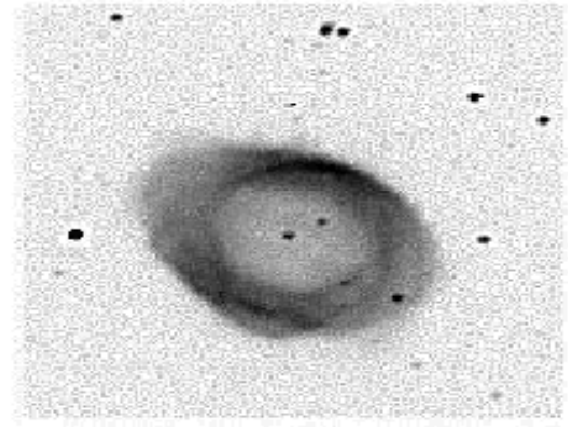
L'application principale des produits de convolution est la création des filtres « passe haut » et « passe bas ». Un filtre « passe haut » favorise les hautes fréquences spatiales, comme les détails, et de ce fait, il améliore le contraste. Un filtre « passe haut » est caractérisé par un noyau comportant des valeurs négatives autour du pixel central, comme dans l'exemple ci-dessous:



*

0	-1	0
-1	5	-1
0	-1	0

=

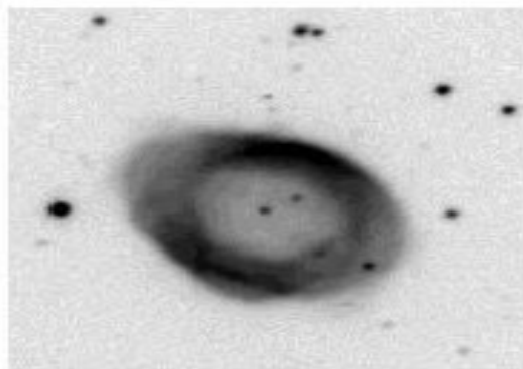


Filtre passe bas

Les filtres « passe bas » agissent en sens inverse des filtres « passe haut » et le résultat est, un adoucissement des détails, ainsi qu'une réduction du bruit granuleux.

Astroart permet la création des filtres de convolution « passe haut » et « passe bas » de n'importe quel type, dans la fenêtre de convolution. Leur utilisation peut-être améliorée à l'aide des autres outils de la fenêtre:

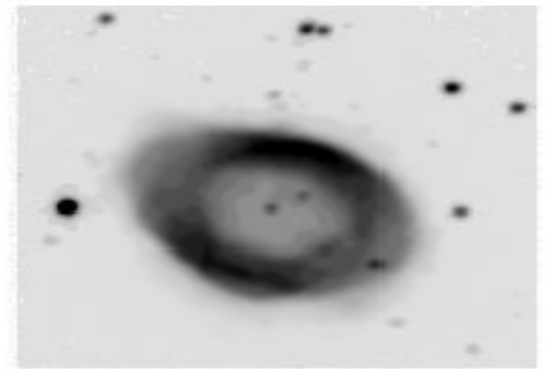
Niveau. Définit une limite maximum (en ADU) de niveau de pixel pour l'application du filtre. Par exemple le filtre passe-bas peut être appliqué dans la zone sombre du fond de ciel, pour réduire le bruit granuleux sans perdre les détails.



*

1	1	1
1	4	1
1	1	1

=



Filtre moyennneur

- Le filtre moyennneur
 - Permet de lisser l'image (*smoothing*)
 - Remplace chaque pixel par la valeur moyenne de ses voisins
 - Réduit le bruit
 - Réduit les détails non-important
 - Brouille ou rend floue l'image (*blur edges*)
- Filtre dont tous les coefficients sont égaux
- Exemple de filtres moyennneurs :

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

ou 1/9

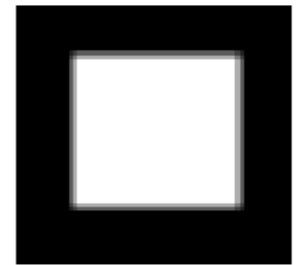
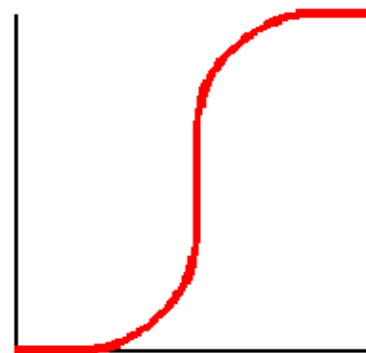
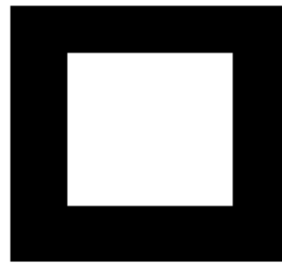
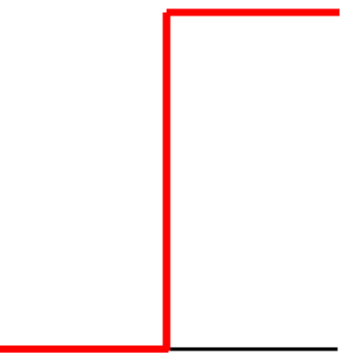
1	1	1
1	1	1
1	1	1

3x3

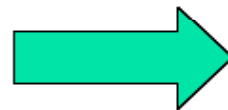
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

5x5

Filtre moyenneur



$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Lissage (*flou apparent*)



*Plus le filtre grossit , plus le lissage devient important et plus le **flou** s'accroît !*

Exemple de filtre moyennneur



Original

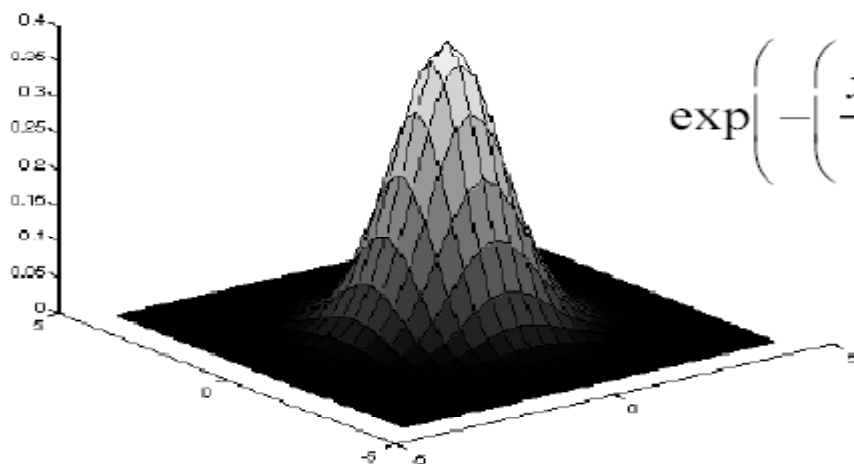


Moyenne 5x5



Moyenne 11x11

Le filtre Gaussien



Fonction gaussienne en 3D

$$\exp\left(-\left(\frac{x^2 + y^2}{2\sigma^2}\right)\right)$$

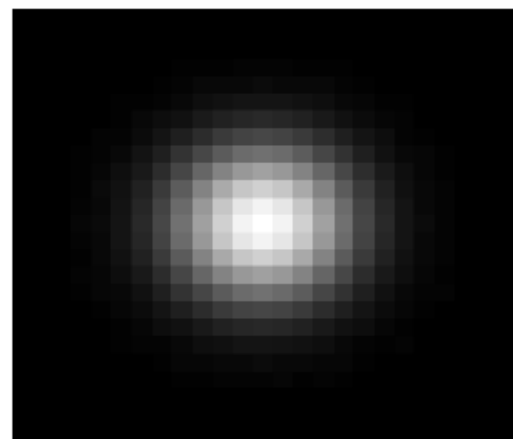
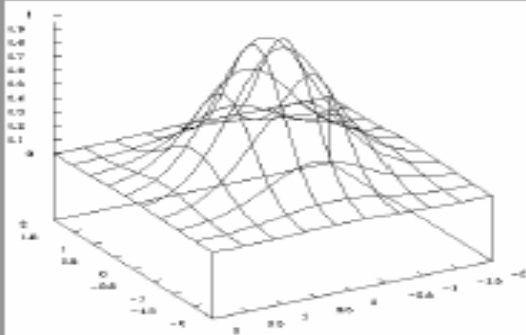


Image d'une gaussienne

Le filtre gaussien donnera un meilleur lissage et une meilleure réduction du bruit que le filtre moyenne

$$\frac{1}{98} \times \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 6 & 8 & 6 & 2 \\ 3 & 8 & 10 & 8 & 3 \\ 2 & 6 & 8 & 6 & 2 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix}$$

Le filtre Gaussien



$$gauss(i, j) = \frac{1}{2\pi\sigma^2} \times e^{-\frac{(i^2 + j^2)}{2\sigma^2}}$$

Fonction gaussienne (normalisée) 2D

Taille du filtre idéal :
 $5\sigma \times 5\sigma$

$1/16 \times$

1	2	1
2	4	2
1	2	1

Cas discret : ($\sigma=0.6$)

$1/1444 \times$

1	9	18	9	1
9	81	162	81	9
18	162	324	162	18
9	81	162	81	9
1	9	18	9	1

Cas discret : ($\sigma=1$)

Exemple de filtre Gaussien



Original



Gauss 5x5



Gauss 11x11

Le filtre pyramidal, conique

- Pyramidal 2D

Filtre pyramidal 1D rapide sur les lignes et sur les colonnes séparément

$$1/81 \times$$

1	2	3	2	1
2	4	6	4	2
3	6	9	6	3
2	4	6	4	2
1	2	3	2	1

- Conique

$$1/25 \times$$

0	0	1	0	0
0	2	2	2	0
1	2	5	2	1
0	2	2	2	0
0	0	1	0	0

- Exemple de combinaison de filtres (passe bas)

$$1/35 \times \begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 2 & 2 & 2 & 1 \\ \hline 1 & 2 & 3 & 2 & 1 \\ \hline 1 & 2 & 2 & 2 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} = 1/35 \times \left\{ \begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} + \begin{array}{|c|} \hline 1 \\ \hline \end{array} \right\}$$

Filtre de prewitt

Les filtres de Prewitt sont les suivants :

Filtre horizontal :

$$h = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

Filtre vertical :

$$v = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

Utilisé principalement pour la détection de contours

Filtre de Sobel

Les filtres de Sobel sont les suivants :

Filtre horizontal :

$$h = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Filtre vertical :

$$v = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & -1 \end{pmatrix}$$

3. Filtrés non linéaires (autre que convolution)

Filtres non linéaires

Deux aspects du lissage sont concernés par le filtrage non linéaire :

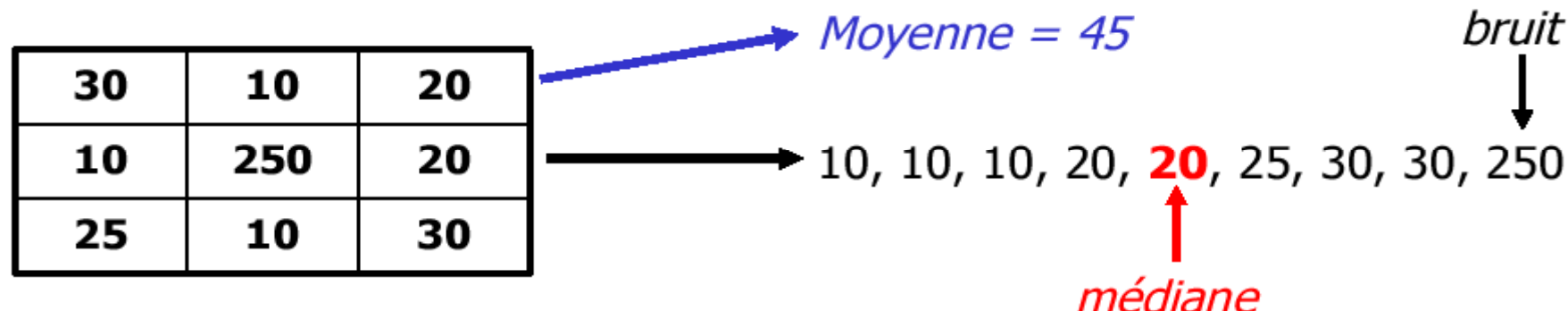
- *Le bruit impulsionnel* : les filtres linéaires éliminent mal les valeurs aberrantes.
- *L'intégrité des frontières* : on souhaiterait éliminer le bruit sans rendre flous les frontières des objets.

On désigne par filtre non linéaire un opérateur qui remplace la valeur de chaque pixel par une combinaison non linéaire des valeurs de ses pixels voisins. L'objectif des filtres lisseurs non linéaires est d'apporter une solution aux limites exprimées ci-dessus, par l'utilisation d'une (ou de plusieurs) des techniques suivantes :

- *statistiques d'ordre* : Opérateurs basés sur un classement des valeurs des pixels voisins (minimum, maximum, médiane, ...)
- *moyennes robustes* : Calculs de moyennes locales - éventuellement pondérées - en excluant du calcul les valeurs marginales.
- *anisotropie* : Opérateurs où le support de calcul du filtre effectivement

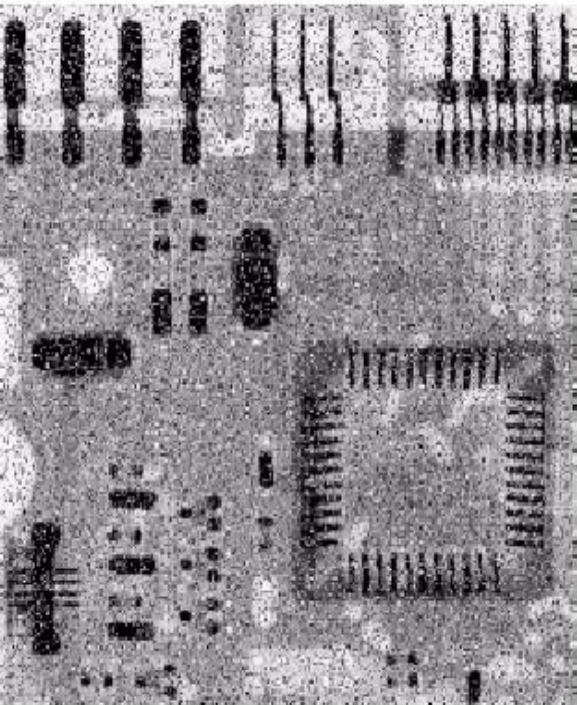
Filtre Median

- Pour nettoyer le bruit dans une image, il existe mieux que le **filtre moyenneur** ou le **filtre gaussien**
- Il s'agit du **filtre médian**
- C'est un filtre non-linéaire, qui ne peut pas s'implémenter comme un produit de convolution
- On remplace la valeur d'un pixel par la valeur médiane dans son voisinage NxN

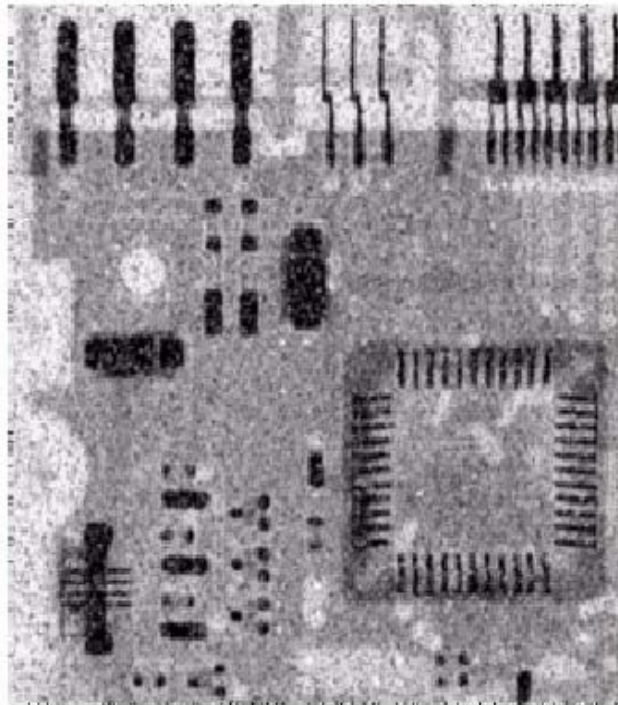


Exemple filtre Median

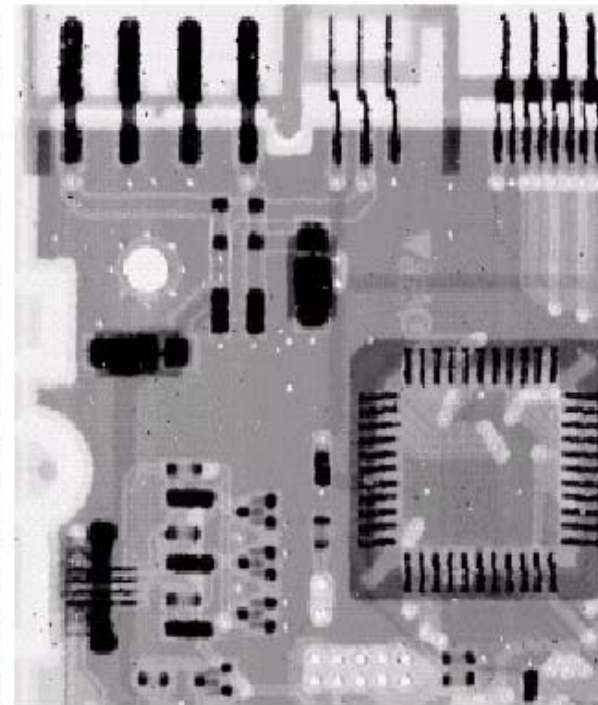
Original



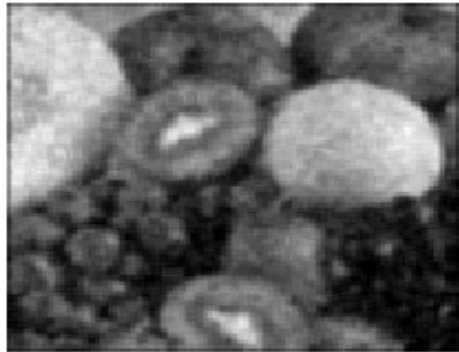
Moyenne 3x3



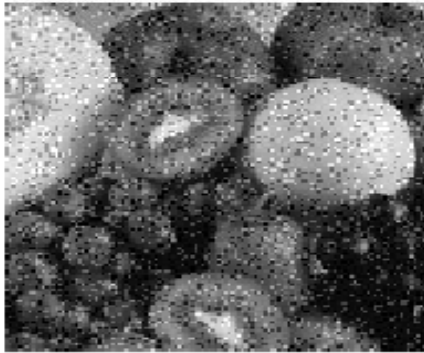
Médian 3x3



Nettoyage du bruit dans une image



3 X 3 Moyenne



Bruit "poivre et sel"



5 X 5 Moyenne

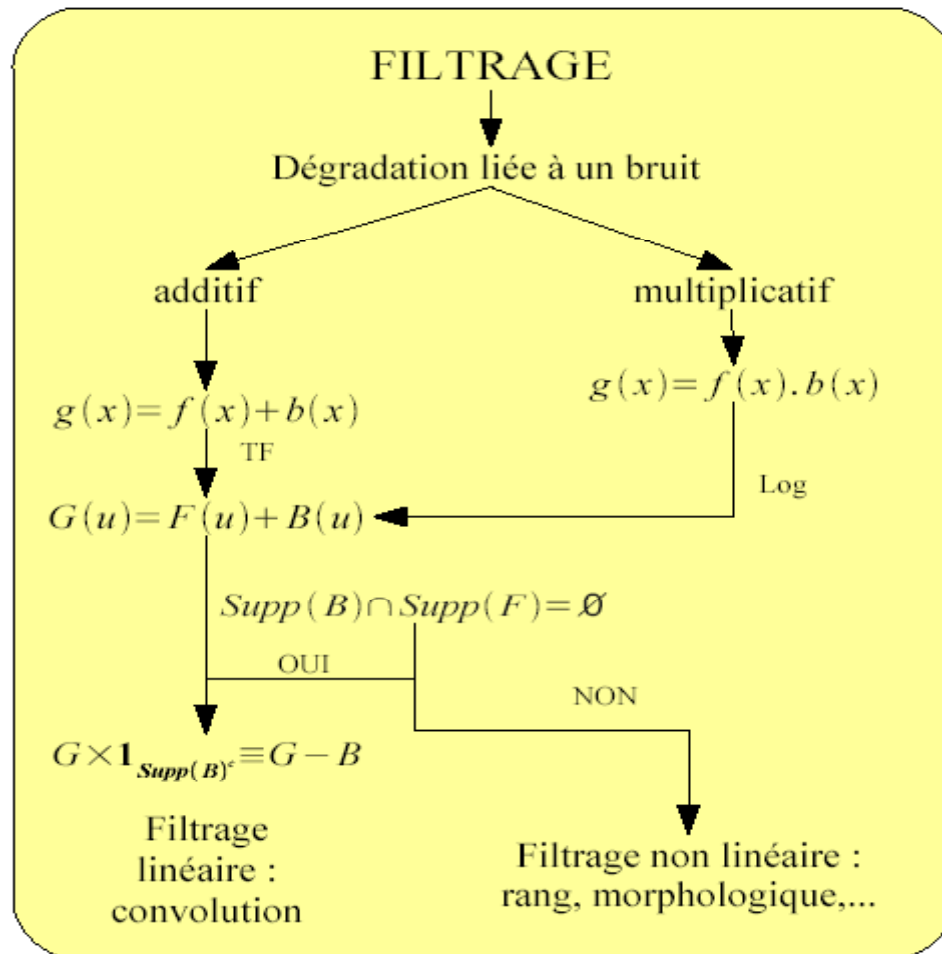


7 X 7 Moyenne



Filtre médian

conclusion



Bibliographie

Caroline Rougier. Cours de Traitement d'images (IFT2730). Université de Montréal (Canada)

<http://www-etud.iro.umontreal.ca/~rougierc/ift2730/>

Chap10 : Filtrage : lissage, réhaussement d'images, détection de contours

http://www-etud.iro.umontreal.ca/~rougierc/ift2730/cours/Cours10_IFT2730_2008_2.pdf

livres :

- J.P. Cocquerez et S. Philipp « *Analyse d'images : filtrage et segmentation* » Masson 1995
- R.C. Gonzalez et Woods « *Digital Image Processing - 2d edition* » Addison Wesley 2002
- A. Rosenfeld et A.C. Kak « *Digital picture processing* » Academic Press - London 1982.
- H. Maître (ss la direction de) « *Le traitement des images* » Hermes Lavoisier IC2 2003.
- J.R. Parker « *Algorithms for Image Processing and Computer Vision* » Wiley & Sons 1997.
- S. Bres, J.M. Jolion, F. Lebourgeois « *Traitement et analyse des images numériques* » Hermes Lavoisier 2003
- I.T. Young, J.J. Gerbrands et L.J. Van Vliet « *Fundamentals of Image Processing* » Université de Delft. (sur internet : <http://www.ph.tn.tudelft.nl/~lucas/publications/1995/FIP95TYJGLV/FIP2.2.pdf>)